

UART + ASCII decoder + stopwatch/Watch

2026.02.10 윤지원

INDEX

목차

✓ 개요

✓ Blockdiagram

✓ FSM, ASM

✓ 핵심코드 및 Simulation

✓ 배운점 및 고찰



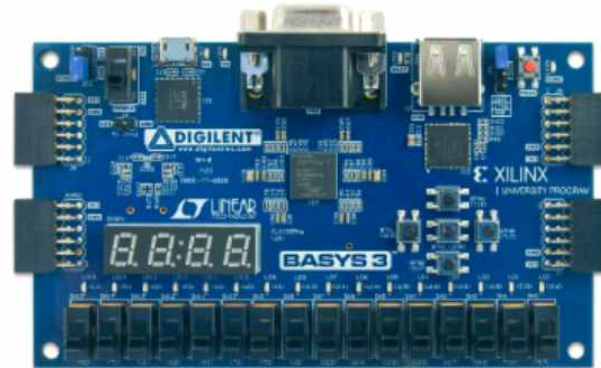
개요

프로젝트 목표

UART 통신과 FPGA 하드웨어 제어를
통합하여 외부 입력으로 조작 가능한
시계 및 스탭워치 시스템 설계

시스템 입력 및 처리

보드의 버튼 또는 스위치 신호와
UART ASCII 데이터 병합 => Tick 신호를 생성

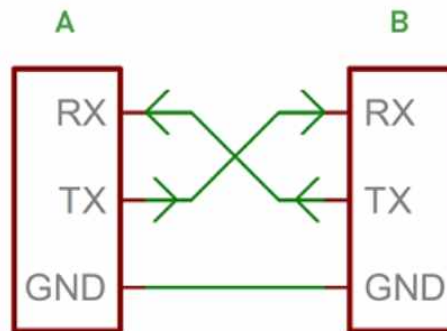


제어 로직 및 상태 관리

FSM과 토글 Toggle 로직

연산 및 데이터 출력

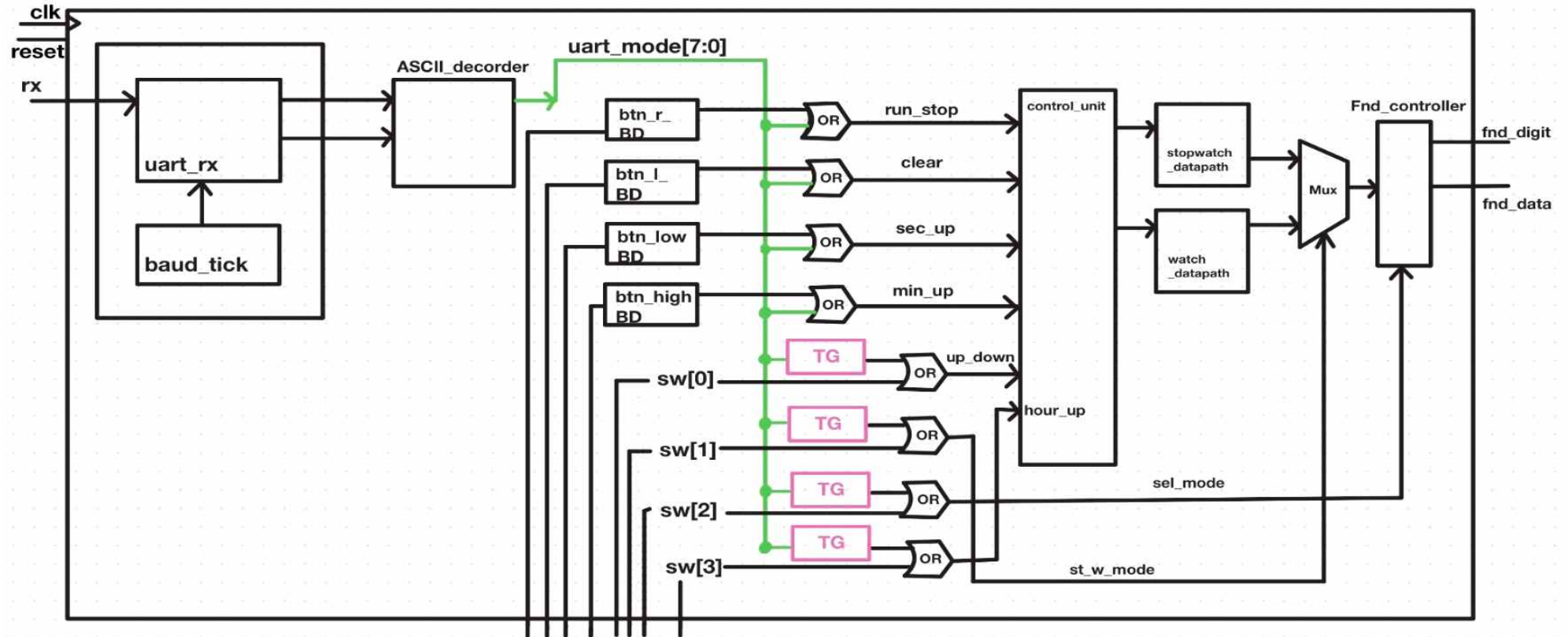
독립적인 두 개의 datapath에서
연산된 시간 데이터
=> MUX
=> FND 컨트롤러
=> 7-Segment



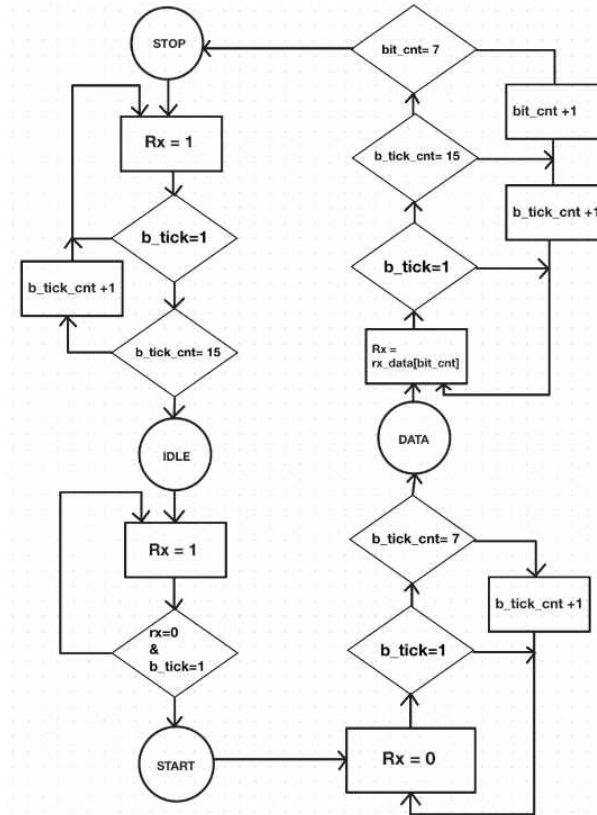
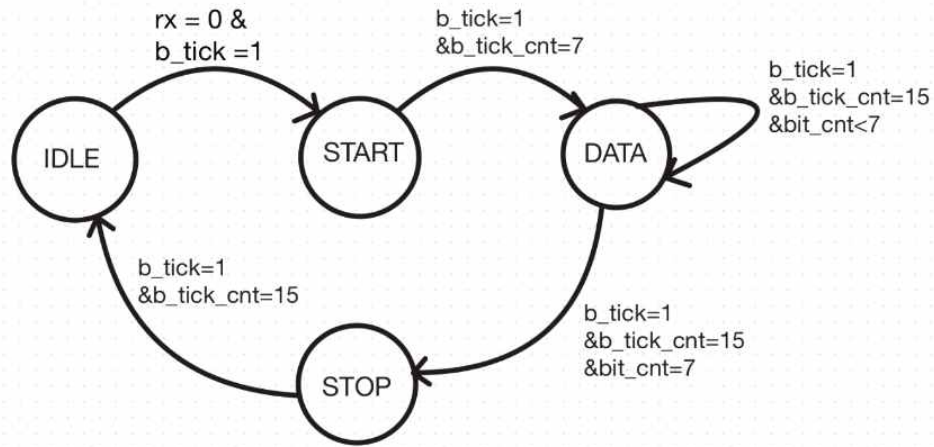
uart 명령키 매핑

UART 명령키	ASCII code (8'h__)	보드	동작 설명
r	72	btn_r	run_stop
l	6c	btn_l	clear
u	75	btn_low	sec_up
d	64	btn_down	min_up
0	30	sw[0]	count up or down
1	31	sw[1]	시계 모드 or 스톱워치 모드
2	32	sw[[2]	시간/분 or 초/밀리초
3	33	sw[3]	hour_up

Blockdiagram

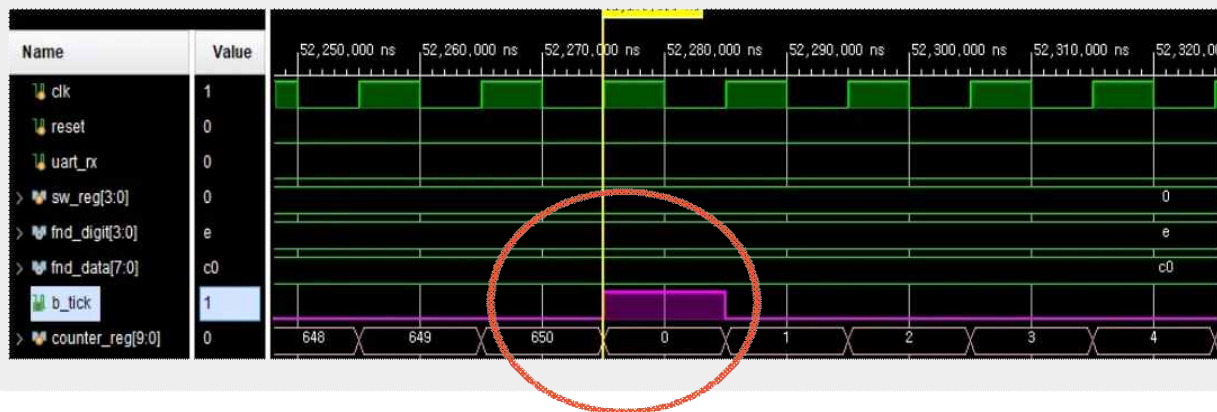


uart => FSM, ASM



baud_tick

```
module baud_tick (  
    input    clk,  
    input    reset,  
    output reg b_tick  
);  
parameter BAUDRATE = 9600*16;  
parameter F_count = 100_000_000 / BAUDRATE;  
reg [flog2(F_count)-1:0] counter_reg;  
  
always @(posedge clk, posedge reset) begin  
    if (reset) begin  
        counter_reg <= 0;  
        b_tick <= 1'b0;  
    end else begin  
        if (counter_reg == (F_count - 1)) begin  
            counter_reg <= 0;  
            b_tick <= 1'b1;  
        end else begin  
            counter_reg <= counter_reg + 1'b1;  
            b_tick <= 1'b0;  
        end  
    end  
end  
endmodule
```



[b_tick 생성 이유]

=> 16배 빠른 tick 사용 => 데이터 비트의 가장 안정적인 지점(가운데)을 찾음

[b_tick 생성 과정]

=> $100_000_000 / (9600 * 16) = 651$ < 원래 주파수 / 목표주파수 >

=> counter_reg = 650 => b_tick (baud_tick) 생성

uart_rx

```
localparam [1:0] IDLE = 2'd0, START = 2'd1, DATA = 2'd2, STOP = 2'd3;
```

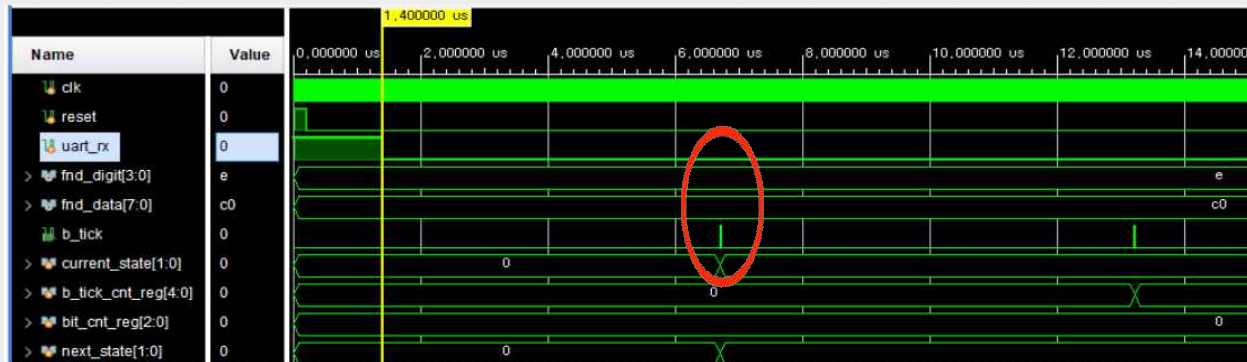
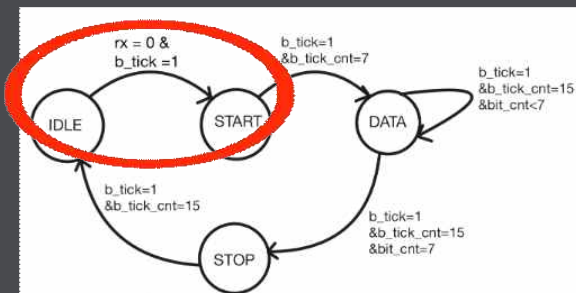
```
always @(*) begin
    next_state = current_state;
    b_tick_cnt_next = b_tick_cnt_reg;
    bit_cnt_next = bit_cnt_reg;
    done_next = done_reg;
    buf_next = buf_reg;
```

```
case (current_state)
```

```
  IDLE: begin
```

```
    done_next = 1'b0;
    b_tick_cnt_next = 5'b0;
    bit_cnt_next = 3'b0;
    if (b_tick & !rx) begin
        next_state = START;
    end
  end
```

```
end
```



[uart_rx 생성 이유]

불규칙한 신호 => FPGA가 이해할 수 있는 **8비트로 정렬** 후 전달

[IDLE => START]

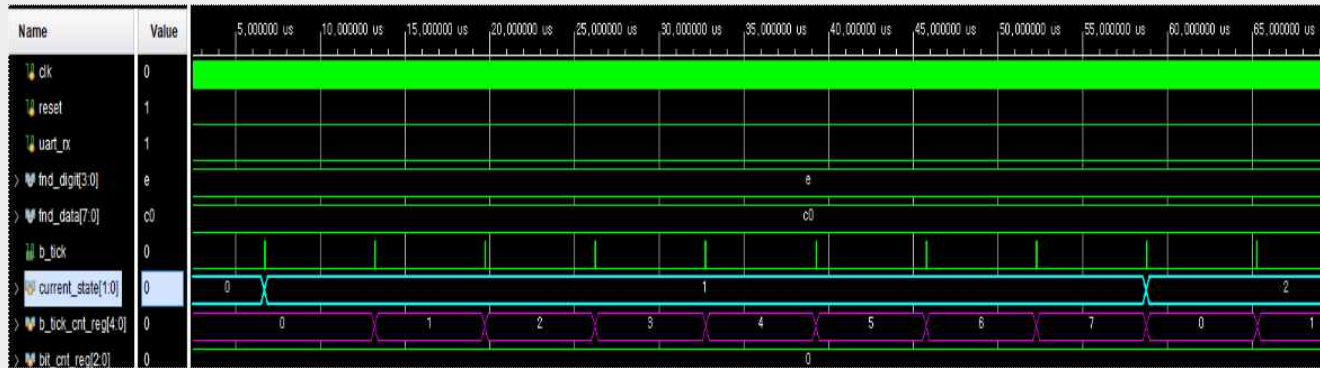
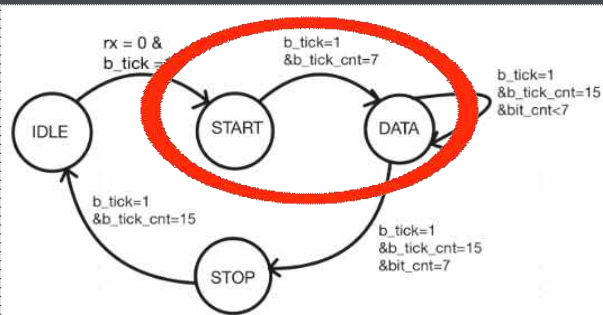
uart_rx : **1 => 0** & b_tick = 1 일때

IDLE => START

uart_rx

```
localparam [1:0] IDLE = 2'd0, START = 2'd1, DATA = 2'd2, STOP = 2'd3;
```

```
START: begin
    if (b_tick)
        if (b_tick_cnt_reg == 7) begin
            b_tick_cnt_next = 0;
            bit_cnt_next = 0;
            next_state = DATA;
        end else begin
            b_tick_cnt_next = b_tick_cnt_reg + 1;
        end
    end
end
```



[START => DATA]

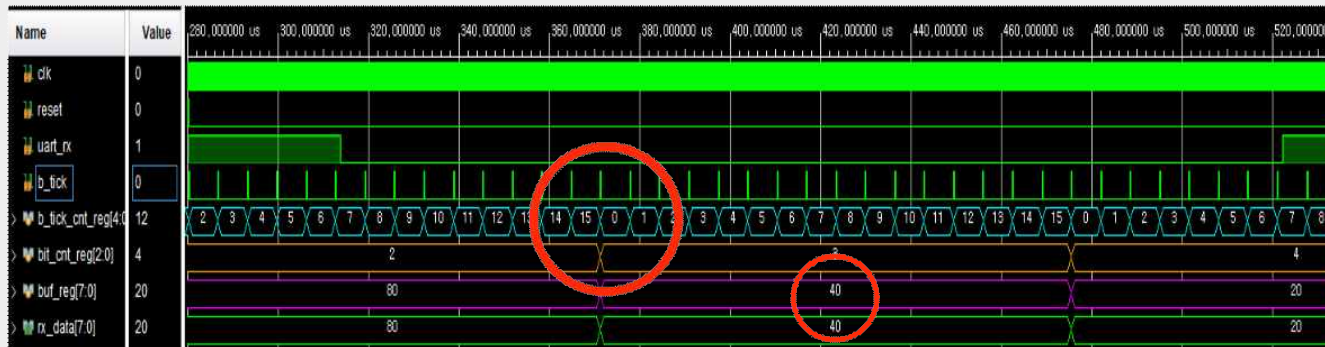
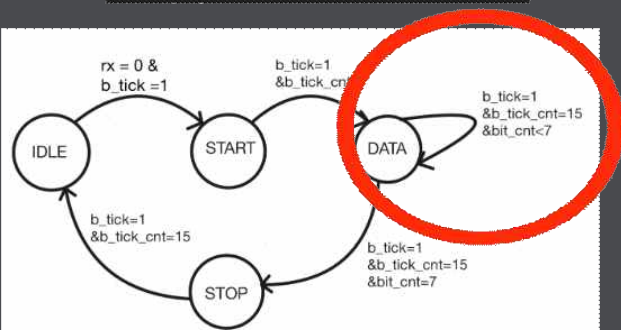
b_tick 발생 => b_tick_cnt ++

b_tick_cnt = 7 => START(1) => DATA(2)

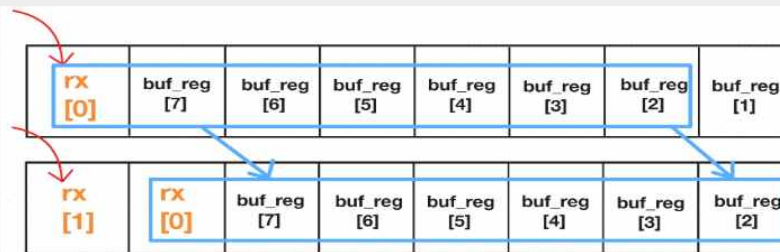
uart_rx

```
localparam [1:0] IDLE = 2'd0, START = 2'd1, DATA = 2'd2, STOP = 2'd3;
```

```
DATA: begin
    if (b_tick) begin
        if (b_tick_cnt_reg == 15) begin
            b_tick_cnt_next = 0;
            buf_next = {rx, buf_reg[7:1]};
            if (bit_cnt_reg == 7) begin
                next_state = STOP;
            end else begin
                bit_cnt_next = bit_cnt_reg + 1;
            end
        end else begin
            b_tick_cnt_next = b_tick_cnt_reg + 1;
        end
    end
end
end
```



[DATA _ shift]



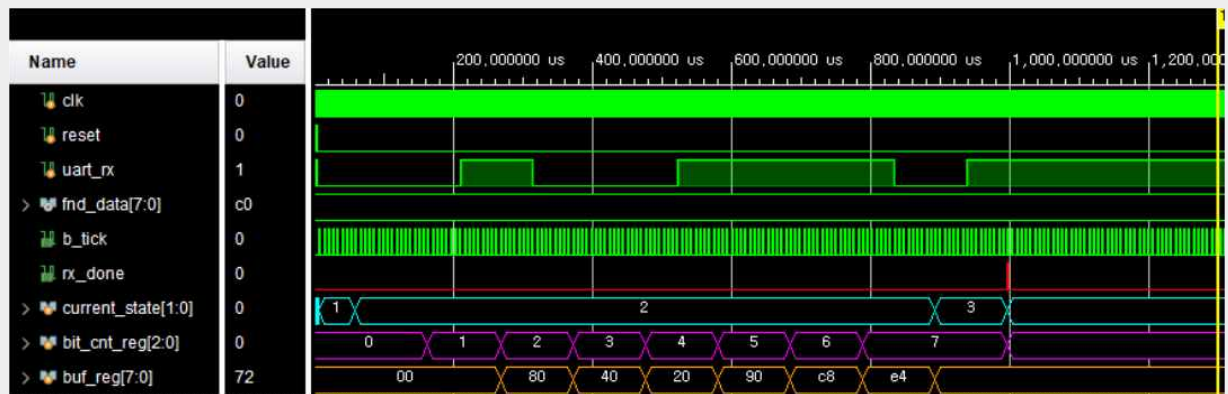
b_tick_cnt_reg = 15 => buf에 shift 된 신호 넣음

uart_rx

```
localparam [1:0] IDLE = 2'd0, START = 2'd1, DATA = 2'd2, STOP = 2'd3;
```

```
DATA: begin
  if (b_tick) begin
    if (b_tick_cnt_reg == 15) begin
      b_tick_cnt_next = 0;
      buf_next = {rx, buf_reg[7:1]};
      if (bit_cnt_reg == 7) begin
        next_state = STOP;
      end else begin
        bit_cnt_next = bit_cnt_reg + 1;
      end
    end else begin
      b_tick_cnt_next = b_tick_cnt_reg + 1;
    end
  end
end
end
```

```
STOP: begin
  if (b_tick)
    if (b_tick_cnt_reg == 15) begin
      next_state = IDLE;
      done_next = 1'b1;
    end else begin
      b_tick_cnt_next = b_tick_cnt_reg + 1;
    end
  end
end
```

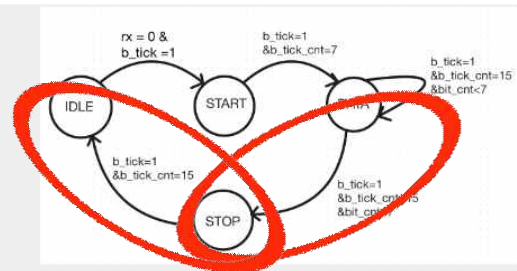


[DATA=> STOP]

bit_cnt_reg = 7 => DATA(2) => STOP(3)

[STOP => IDLE]

b_tick_cnt_reg = 15 => STOP(3) => IDLE(0) => done 1tick 출력



ASCII_decoder

```

module ascii_decoder (
    input      clk,
    input      reset,
    input [7:0] uart_data,
    input      uart_done,
    output reg [7:0] uart_mode
);
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            uart_mode <= 8'b0;
        end else begin
            if (uart_done) begin
                case (uart_data)
                    8'h72: uart_mode <= 8'h01; // 'r'
                    8'h6c: uart_mode <= 8'h02; // 'l'
                    8'h75: uart_mode <= 8'h04; // 'u'
                    8'h64: uart_mode <= 8'h08; // 'd'
                    8'h30: uart_mode <= 8'h10; // '0'
                    8'h31: uart_mode <= 8'h20; // '1'
                    8'h32: uart_mode <= 8'h40; // '2'
                    8'h33: uart_mode <= 8'h80; // '3'
                    default: uart_mode <= 8'b0;
                endcase
            end else begin
                uart_mode <= 8'b0;
            end
        end
    end
endmodule

```



[ASCII(8bit) => 8bit 값 tick]
 ex) 'r' : RUN이 눌렸을 경우

uart_done=1
 일 때
 uart_data (72) : run 'r'(01)

-----> 'r' 에 매핑
 uart_mode = 01

sw_toggle

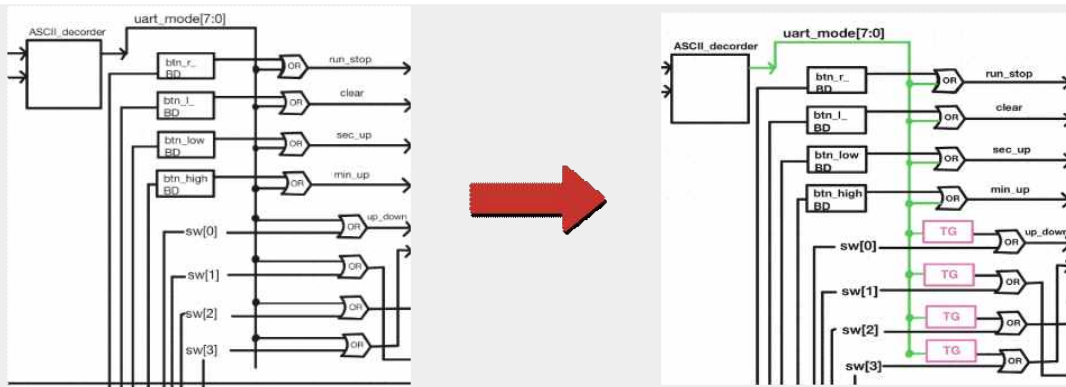
```
module sw_toggle (  
    input    clk,  
    input    reset,  
    input    tick,  
    output reg state  
);  
always @(posedge clk or posedge reset) begin  
    if (reset) begin  
        state <= 1'b0;  
    end else if (tick) begin  
        state <= ~state;  
    end  
end  
endmodule
```

[Trouble Shooting]

문제점 :

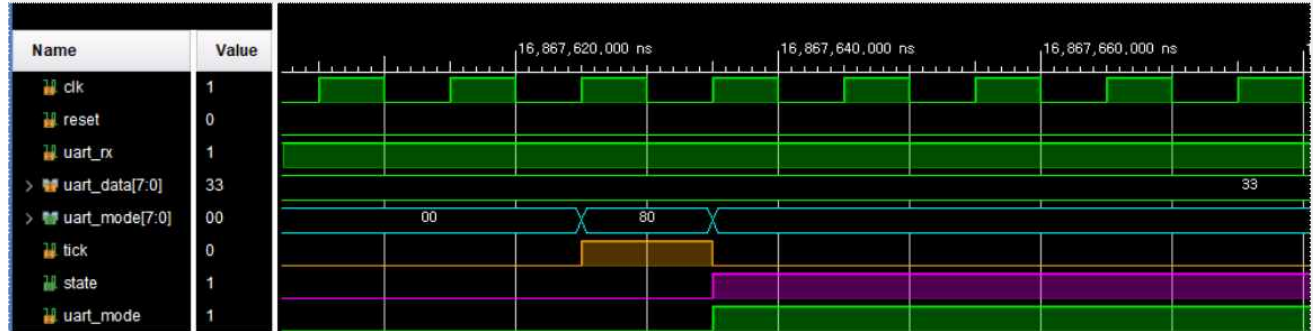
Ascii_decorder에서 나온 uart_tick 신호를 하드웨어의 sw 와 btn 와 같이 or_gate에 넣었을때 uart_btn 신호만 잘 인식하고 uart_sw 신호는 인식 하지 못함

해결법 : decorder의 **tick** 신호 => **toggle** 신호 로 변환



sw_toggle

```
module sw_toggle (  
    input    clk,  
    input    reset,  
    input    tick,  
    output reg state  
);  
    always @(posedge clk or posedge reset) begin  
        if (reset) begin  
            state <= 1'b0;  
        end else if (tick) begin  
            state <= ~state;  
        end  
    end  
endmodule
```



입력 : (decoder) **tick** => 출력 : state **toggle**

state : 0 => tick => state: 1

=> 이후 toggle의 출력값은 or_gate로 전달

or_gate

```
module or_gate (  
    input  uart_mode,  
    input  orig_mode,  
    output select_mode  
);  
    assign select_mode = uart_mode | orig_mode;  
endmodule
```



uart_mode (ascii_decorder의 출력)와 orig_mode (하드웨어 신호)

둘중 하나라도 1이면 1 출력

설계 이유 : 하드웨어와 uart에서 모두 보드를 작동시키기 위함

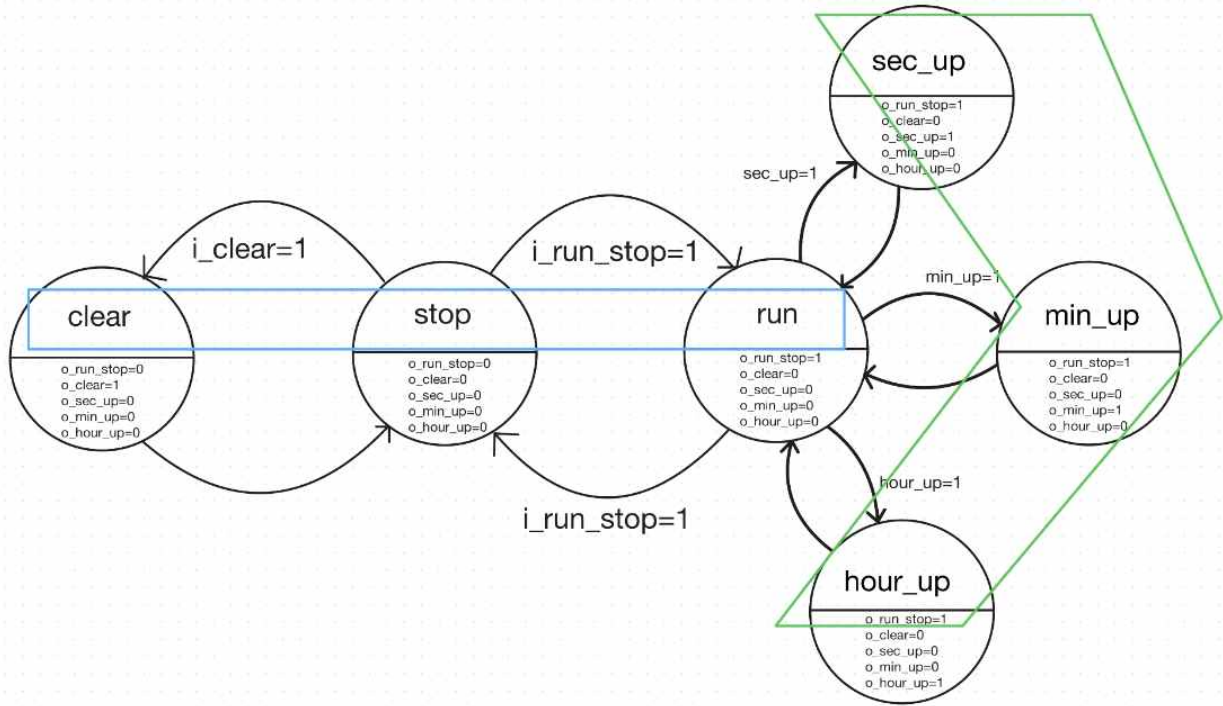
control_unit

```

always @(*) begin
    next_st = current_st;
    o_run_stop = 1'b0;
    o_clear = 1'b0;
    o_sec_up = 1'b0;
    o_min_up = 1'b0;
    o_hour_up = 1'b0;
    case (current_st)
        STOP: begin
            // no output
            o_run_stop = 1'b0;
            o_clear = 1'b0;
            if (i_run_stop) begin
                next_st = RUN;
            end else if (i_clear) begin
                next_st = CLEAR;
            end
        end
        RUN: begin
            o_run_stop = 1'b1;
            if (i_run_stop) begin
                next_st = STOP;
            end else if (i_sec_up) begin
                next_st = SEC_UP;
            end else if (i_min_up) begin
                next_st = MIN_UP;
            end else if (i_hour_up_tick) begin
                next_st = HOUR_UP;
            end
        end
        CLEAR: begin
            o_clear = 1'b1;
            next_st = STOP;
        end
        SEC_UP: begin
            o_sec_up = 1'b1;
            next_st = RUN;
        end
        MIN_UP: begin
            o_min_up = 1'b1;
            next_st = RUN;
        end
        HOUR_UP: begin
            o_hour_up = 1'b1;
            next_st = RUN;
        end
    endcase
end

```

FSM



control_unit

```
RUN: begin
  o_run_stop = 1'b1;
  if (i_run_stop) begin
    next_st = STOP;
  end else if (i_sec_up) begin
    next_st = SEC_UP;
  end else if (i_min_up) begin
    next_st = MIN_UP;
  end else if (i_hour_up) begin
    next_st = HOUR_UP;
  end
end
```

Wire w_hour_up_tick = i_hour_up & ~r_hour_up_prev;

```
always @(posedge clk or posedge reset) begin
  if (reset) begin
    current_st <= STOP;
    r_hour_up_prev <= 1'b0;
  end else begin
    current_st <= next_st;
    r_hour_up_prev <= i_hour_up;
  end
end
```

```
RUN: begin
  o_run_stop = 1'b1;
  o_clear = 1'b0;
  o_sec_up = 1'b0;
  o_min_up = 1'b0;
  o_hour_up = 1'b0;
  if (i_run_stop) begin
    next_st = STOP;
  end else if (i_sec_up) begin
    next_st = SEC_UP;
  end else if (i_min_up) begin
    next_st = MIN_UP;
  end else if (w_hour_up_tick) begin
    next_st = HOUR_UP;
  end
end
```

[Trouble Shooting]

문제점 :

i_hour_up 으로 했을때 실제 하드웨어에서 hour_up 값이 랜덤으로 숫자 올라감

원인:

hour_up = 1 일때 계속 상승하라는 의미로 전달 => 값 계속 증가

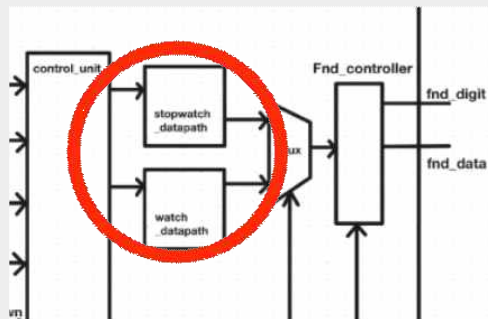
hour_up은 스위치 신호 (값이 한번들어오면 다음신호 들어올때까지 유지)

해결법 :

스위치를 올린 상태로 두어도 시간이 딱 한 번만 올라가도록

(0) => (1)으로 변하는 그 순간 저장

datapath



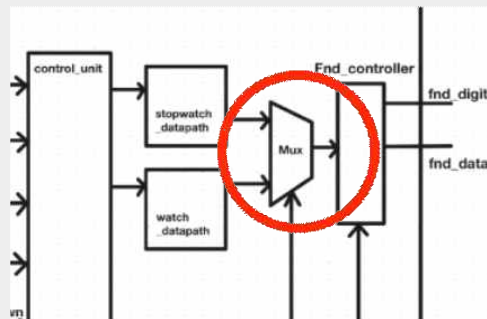
< 역할 >

control_unit의 명령을 받아서
실제로 데이터를 처리, 연산, 저장

< stopwatch_datapath (24bit) > :
run_stop, clear, up_down

< watch_datapath (24bit) > :
up_down, sec_up, min_up, hour_up

mux



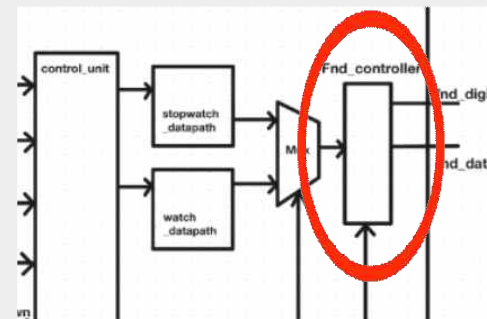
< 역할 >

sw[1]의 값에 따라 stopwatch or watch 선택

sw [1] = 0 => stopwatchtime

sw [1] = 1 => watchtime

fnd_controller



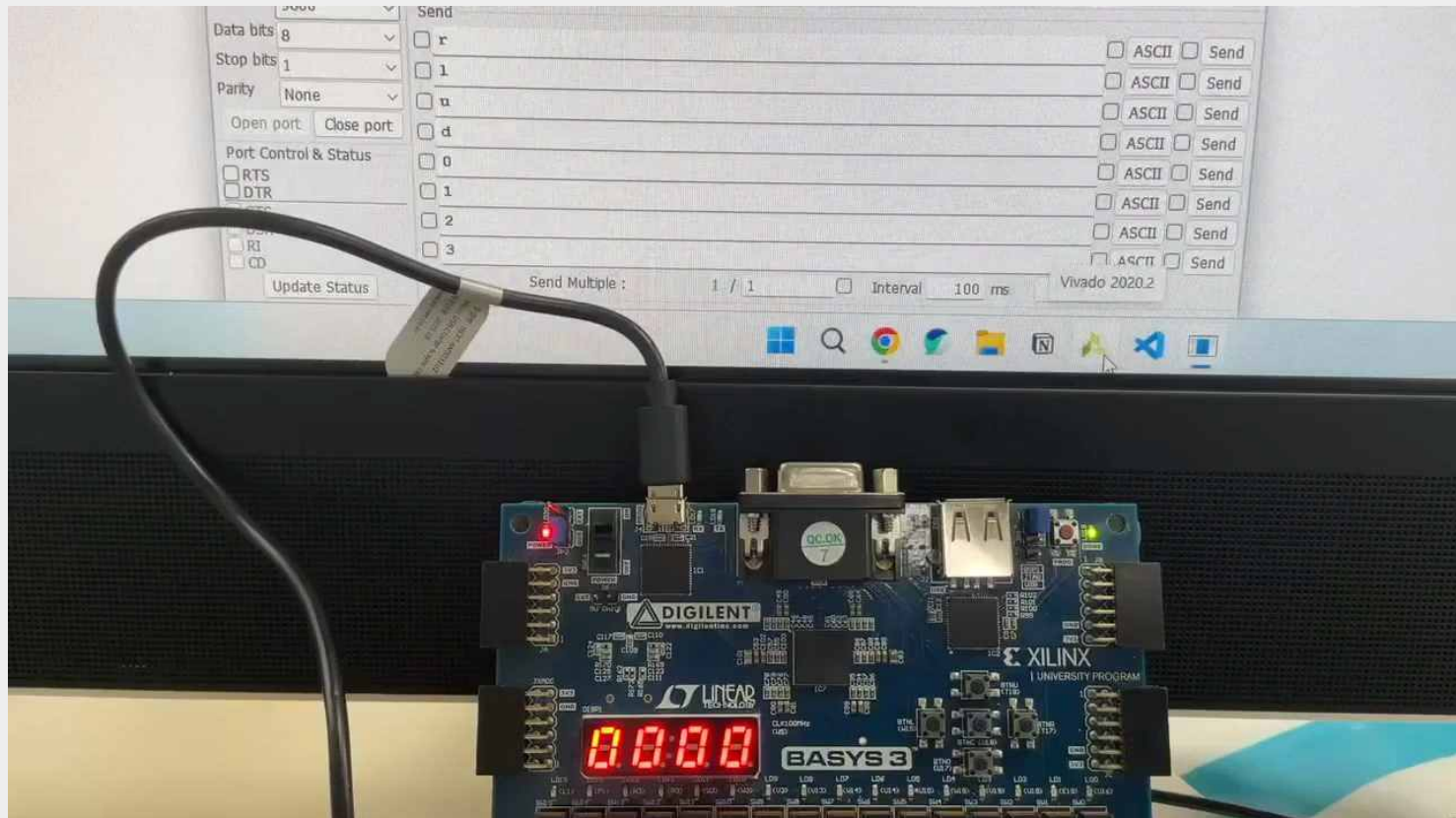
< 역할 >

26bit 시간 데이터를 사람이 눈으로 읽을 수 있게
fnd에 순차적으로 뿌려줌

< 처리 >

데이터를 자릿수별로 쪼개고 (splitter)
1ms마다 자릿수를 변경 (clk_div)
그에 맞는 숫자 선택 (mux)

시연 영상



배운점 및 고찰

ascii 의 16진수 값을 잘못 대입

c(63) => l(6c)

```
8'h63:  uart_mode <= 8'h02;  // 'c'
```



```
8'h6c:  uart_mode <= 8'h02;  // 'l'
```

uart_mode	
☐	7
☒	1
☐	0
☐	4
☐	0
☐	1
☐	2
☐	3

좀 더 꼼꼼함 필요

clear 의 간헐적 오류

uart 의clear는 잘 처리

But 하드웨어 동작시 갑자기 안 먹는 경우 발생

이럴경우 reset 을 누르면 다시 잘 작동

<미해결>

원인 예상 : btn_l 에 더 많은 노이즈 섞여있음



Thank You

감사합니다